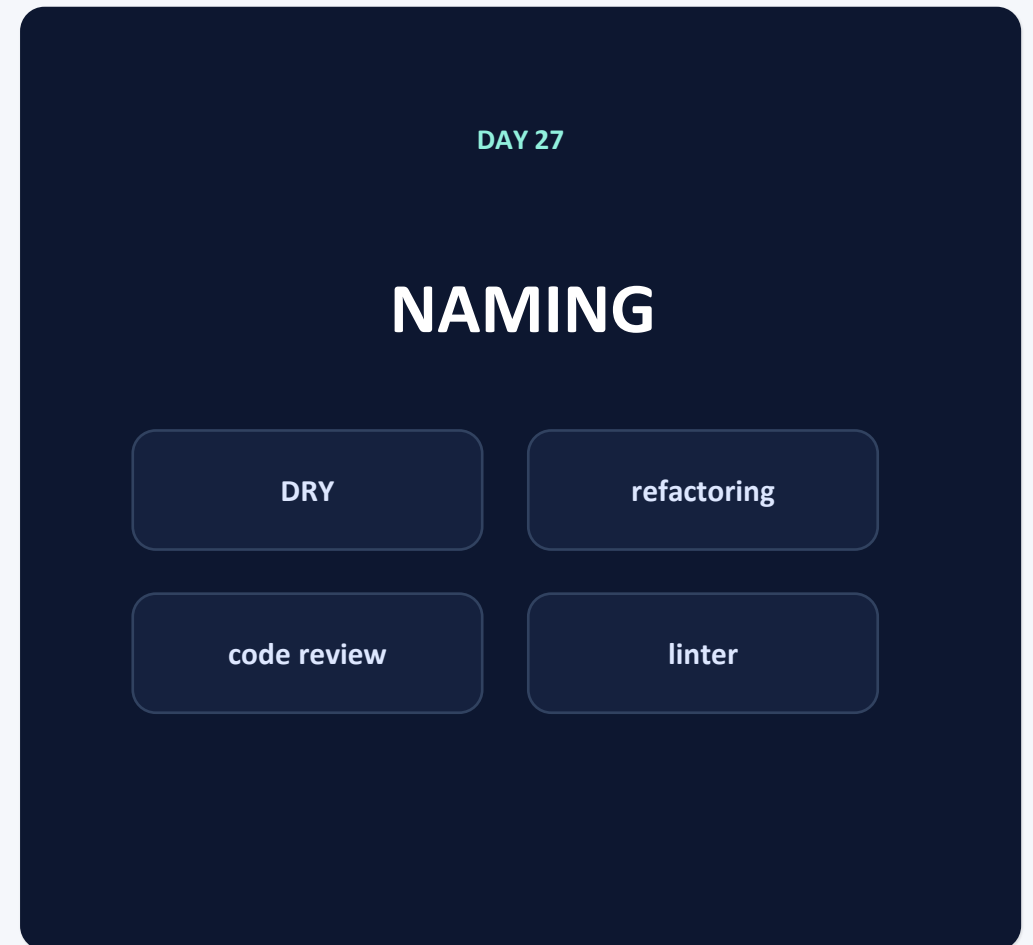


Чистый код и code review

Делать код понятнее через хорошие имена, маленькие функции, DRY, рефакторинг, форматтер/линтер и конструктивный code review.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

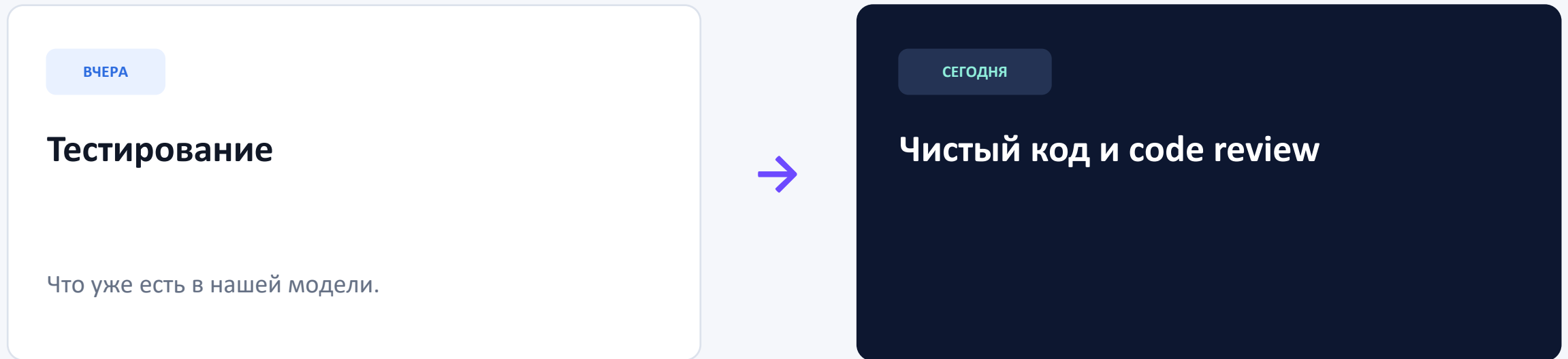
Конкретные действия вместо списка терминов.

- 01 Делать код понятнее через хорошие имена, маленькие функции, DRY, рефакторинг, форматтер/линтер и конструктивный code review.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.



Вчера: Тестирование. Сегодня мы добавляем следующий слой: чистый код и code review.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Код читают намного чаще, чем пишут. Понятность уменьшает количество ошибок и ускоряет совместную работу.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Чистый код похож на хорошо организованную кухню: каждый инструмент на своём месте, названия понятны, повторяющиеся действия вынесены в отдельные процессы.

ПЕРЕВОД В IT

naming

выбор имён, раскрывающих назначение данных и действий

DRY

принцип устранения повторяющегося знания

refactoring

улучшение структуры без изменения поведения

Слой 1: naming • DRY • refactoring

Сначала смысл, затем профессиональный термин.

**naming**

выбор имён, раскрывающих назначение данных и действий

**DRY**

принцип устранения повторяющегося знания

**refactoring**

улучшение структуры без изменения поведения

Слой 2: code review • linter • technical debt

Сначала смысл, затем профессиональный термин.

**code review**

проверка изменений другим разработчиком

**linter**

автоматический анализатор стиля и типовых ошибок

**technical debt**

будущая стоимость принятых сегодня упрощений

ТЕОРИЯ

Слой 3: readability • duplication

Сначала смысл, затем профессиональный термин.

**readability**

насколько легко человеку понять код

**duplication**

повторение одинаковой логики в нескольких местах

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: code smell • formatter

Сначала смысл, затем профессиональный термин.

code smell

признак потенциальной проблемы в устройстве кода

formatter

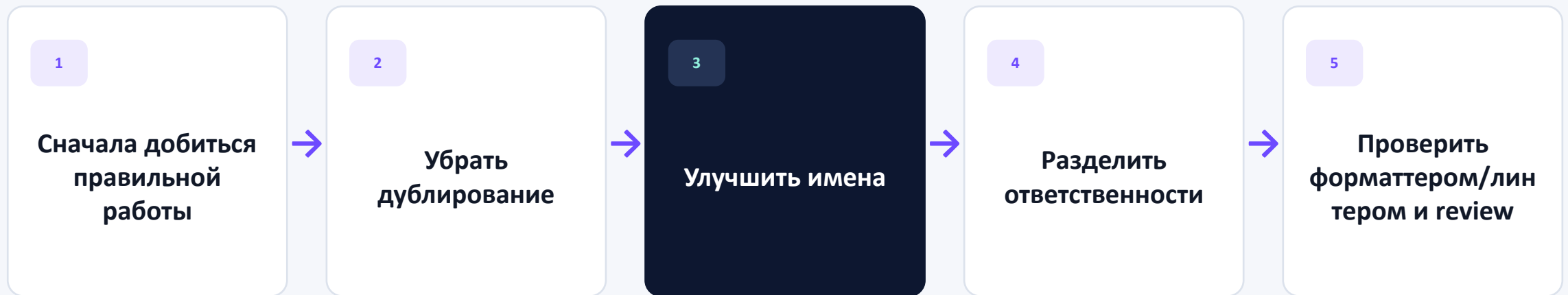
инструмент автоматического единообразного оформления кода

Не заучивай формулировку. Придумай свой пример использования каждого термина.

СХЕМА

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day27_step1

```
def calculate_total(price, quantity):  
    return price * quantity  
  
print(calculate_total(10, 3))
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
def calculate_total(price, quantity):
```

Создаём функцию и даём ей имя; код внутри выполнится при вызове.

```
    return price * quantity
```

Python выполняет эту строку сверху вниз.

```
print(calculate_total(10, 3))
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
def calculate_total(price, quantity):  
    return price * quantity  
  
print(calculate_total(10, 3))
```

РЕЗУЛЬТАТ

30

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
VAT_RATE = 0.20
```

```
def add_vat(price):  
    return price * (1 + VAT_RATE)
```

```
print(add_vat(100))
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

120.0

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.



```
def f(a, b):  
    x = a * b  
    return x
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

Код работает, но имена ничего не объясняют. Читателю приходится угадывать смысл каждого значения.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

```
def calculate_total(price, quantity):  
    total = price * quantity  
    return total
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Отрефакторить расчёт цены с НДС, вынеся процент в именованную константу и вычисление в функцию.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Сохраняем поведение прежним.**
- 2 Даём коэффициенту имя VAT_RATE.
- 3 Выносим расчёт в add_vat.
- 4 Функция получает только нужный аргумент.
- 5 Проверяем результат до/после.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.



solution_part1

```
VAT_RATE = 0.20

def add_vat(price):
    return price * (1 + VAT_RATE)
```

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



```
VAT_RATE = 0.20

def add_vat(price):
    return price * (1 + VAT_RATE)

prices = [10, 20]
for price in prices:
    print(add_vat(price))
```

solution_final

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

12.0
24.0

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Улучши код `d = p * q; print(d)` так, чтобы смысл был понятен без комментария.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Начни с имён price, quantity, total.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
price = 15  
quantity = 4  
total = price * quantity  
print(total)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

60

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую законченную задачу.

ЗАДАНИЕ

Найди дублирование двух одинаковых расчётов скидки и вынеси его в функцию.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



```
def discounted(price, percent):  
    return price * (1 - percent / 100)  
  
print(discounted(100, 10))  
print(discounted(250, 10))
```

mini_project

РЕЗУЛЬТАТ ПРОЕКТА

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

90.0
225.0

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Гнаться за «умным» коротким кодом вместо ясности.

02

Рефакторить и менять поведение одновременно без тестов.

03

Создавать функции ради каждой одной строки без смысла.

04

Воспринимать review как личную критику.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

- 1 Что такое «naming» и какую проблему это понятие помогает описать?
- 2 Чем «naming» отличается от «code review»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **naming — выбор имён, раскрывающих назначение данных и действий.**

2 **naming: выбор имён, раскрывающих назначение данных и действий. code review: проверка изменений другим разработчиком.**

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

³ **30**

⁴ **Код работает, но имена ничего не объясняют. Читателю приходится угадывать смысл каждого значения.**

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ **Делать код понятнее через хорошие имена, маленькие функции, DRY, рефакторинг, форматтер/линтер и конструктивный code review.**
- ✓ Читать и объяснять пример темы «Чистый код и code review».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Безопасность приложений

Завтра: Безопасность приложений. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.